

Minimal DSA Topics (2-Month Plan)

1. Basics (Week 1)

- What is DSA and why it matters
- Time & Space Complexity
- Big O Notation (focus on $O(1)$, $O(n)$, $O(\log n)$, $O(n^2)$)
- Recursion (factorial, Fibonacci, sum of array)

Practice:

1. Write a function to calculate factorial using recursion.
 2. Find the sum of elements in an array using recursion.
 3. Print the Fibonacci sequence using recursion.
-

2. Arrays (Week 2)

- Declaration & Traversal
- Insertion and Deletion
- Searching (Linear & Binary Search)
- Sorting (Bubble, Insertion)
- Simple array problems:
 - Reverse array
 - Find max/min
 - Sum of array
 - Two-sum (pair with given sum)

Practice:

1. Find the maximum and minimum element in an array.
2. Search an element using binary search.
3. Sort an array using insertion sort.



3. Strings (Week 3)

- Basic string manipulation (length, compare, reverse)
- Palindrome check
- Count vowels/consonants
- Word frequency (using arrays or maps)

Practice:

1. Check if a string is a palindrome.
 2. Count vowels and consonants in a string.
 3. Find the frequency of each character in a string.
-



4. Linked List (Week 4)

- Singly Linked List (create, insert, delete, traverse)
- Reverse a linked list
- Find middle element
- Detect loop (basic understanding)

(Skip doubly & circular for now — focus on singly only.)

Practice:

1. Create a singly linked list and display all nodes.
 2. Reverse a linked list.
 3. Find the middle element of a linked list.
-

5. Stack & Queue (Week 5)

- **Stack:**
 - Push, Pop, Peek
 - Applications: balanced parentheses, postfix evaluation
- **Queue:**
 - Enqueue, Dequeue
 - Simple applications (process tasks in order)

Practice:

1. Implement a stack using arrays.
 2. Check if parentheses are balanced in an expression.
 3. Implement a queue using arrays
-

6. Trees (Week 6)

- Binary Tree Basics
- Tree Traversals (Inorder, Preorder, Postorder)
- Binary Search Tree (insert, search)

(Skip AVL, Heaps for now — just understand the concept.)

Practice:

1. Create a binary tree and print its inorder traversal.
 2. Insert elements into a BST and search for a value.
 3. Find the height of a binary tree.
-

7. Searching & Sorting Algorithms (Week 7)

- Binary Search (recursive & iterative)
- Merge Sort
- Quick Sort
- Compare complexities

Practice:

1. Perform binary search on a sorted array (recursive).
 2. Implement merge sort.
 3. Implement quick sort.
-

8. Graphs (Week 8)

- Basic Graph Representation (Adjacency List & Matrix)
- BFS and DFS traversal

Practice:

1. Represent a graph using an adjacency list.
2. Perform BFS traversal on a simple graph.
3. Perform DFS traversal on the same graph.